```
import math
import random
import numpy as np


# ---------------------------------------------------------------------------
# 1. WSN Energy Objective Function
# ---------------------------------------------------------------------------
def wsn_energy_fitness(nest, num_nodes, area_size, sink_pos, E_initial):
  """
    Evaluates the fitness of a candidate nest.
    In WSN clustering/routing optimization, a nest can represent
    Cluster Head (CH) selections or energy weights.
    Here, lower energy consumption / longer network lifetime is optimal.
    """
  # Ensure nest values are bounded/normalized
  total_energy_consumed = 0.0

  # Example WSN energy dissipation model (First-Order Radio Model approximation)
  E_elec = 50e-9  # Energy per bit to run transmitter or receiver (J/bit)
  E_fs = 10e-12  # Free space amp model (J/bit/m^2)
  E_mp = 0.0013e-12  # Multipath fading amp model (J/bit/m^4)
  d0 = math.sqrt(E_fs / E_mp)
  k_bits = 4000  # Packet size in bits

  for i in range(num_nodes):
    # Simulated distance to sink or cluster head based on nest values
    node_x, node_y = nest[i * 2], nest[i * 2 + 1]
    dist_to_sink = math.sqrt(
        (node_x - sink_pos[0]) ** 2 + (node_y - sink_pos[1]) ** 2
    )

    if dist_to_sink < d0:
      E_tx = k_bits * E_elec + k_bits * E_fs * (dist_to_sink**2)
    else:
      E_tx = k_bits * E_elec + k_bits * E_mp * (dist_to_sink**4)

    total_energy_consumed += E_tx

  # We want to minimize energy consumption (fitness = lower energy)
  return total_energy_consumed


# ---------------------------------------------------------------------------
# 2. Levy Flight Generator
# ---------------------------------------------------------------------------
def levy_flight(beta=1.5):
  sigma = (
      math.gamma(1 + beta)
      * math.sin(math.pi * beta / 2)
      / (
          math.gamma((1 + beta) / 2)
          * beta
          * math.pow(2, (beta - 1) / 2)
      )
  ) ** (1.0 / beta)
  u = np.random.normal(0, sigma, 1)[0]
  v = np.random.normal(0, 1, 1)[0]
  step = u / (abs(v) ** (1.0 / beta))
  return step


# ---------------------------------------------------------------------------
# 3. Cuckoo Search Algorithm for WSN
# ---------------------------------------------------------------------------
def cuckoo_search_wsn(
    num_nodes,
    area_size,
    sink_pos,
    pop_size,
    max_iter,
    pa=0.25,
    E_initial=0.5,
):
  dim = num_nodes * 2  # Each node has (x, y) coordinates
  # Initialize nests randomly within the area bounds [0, area_size]
  nests = np.random.uniform(0, area_size, (pop_size, dim))
  fitness = np.array(
      [
          wsn_energy_fitness(nest, num_nodes, area_size, sink_pos, E_initial)
          for nest in nests
```

```
        for nest in nests
      ]
  )

  best_idx = np.argmin(fitness)
  best_nest = np.copy(nests[best_idx])
  best_fitness = fitness[best_idx]

  for iteration in range(max_iter):
    # Generate new solutions (cuckoos) via Levy flights
    for i in range(pop_size):
      new_nest = np.copy(nests[i])
      alpha = 0.01
      step = levy_flight(beta=1.5)
      new_nest += alpha * step * (new_nest - best_nest)

      # Bound constraints [0, area_size]
      new_nest = np.clip(new_nest, 0, area_size)

      new_fitness = wsn_energy_fitness(
          new_nest, num_nodes, area_size, sink_pos, E_initial
      )

      # Replace if better
      j = random.randint(0, pop_size - 1)
      if new_fitness < fitness[j]:
        nests[j] = new_nest
        fitness[j] = new_fitness

    # Abandon a fraction (pa) of worse nests and build new ones
    for i in range(pop_size):
      if random.random() < pa:
        nests[i] = np.random.uniform(0, area_size, dim)
        fitness[i] = wsn_energy_fitness(
            nests[i], num_nodes, area_size, sink_pos, E_initial
        )

    # Find the current best
    current_best_idx = np.argmin(fitness)
    if fitness[current_best_idx] < best_fitness:
      best_fitness = fitness[current_best_idx]
      best_nest = np.copy(nests[current_best_idx])

    if (iteration + 1) % max(1, (max_iter // 5)) == 0 or iteration == 0:
      print(f"Iteration {iteration+1}/{max_iter} - Best Energy: {best_fitness:.6f} J")

  return best_nest, best_fitness


# --------------------------------------------------------------------------
# 4. User Input Handler
# --------------------------------------------------------------------------
if __name__ == "__main__":
  print("=== Wireless Sensor Network (WSN) Cuckoo Search Optimization ===")
  try:
    num_nodes = int(input("Enter number of sensor nodes (e.g., 10): ") or 10)
    area_size = float(
        input("Enter WSN deployment area side length in meters (e.g., 100): ")
        or 100.0
    )
    sink_x = float(input("Enter Sink X-coordinate (e.g., 50): ") or 50.0)
    sink_y = float(input("Enter Sink Y-coordinate (e.g., 50): ") or 50.0)
    sink_pos = (sink_x, sink_y)
    pop_size = int(
        input("Enter Cuckoo Search population size / number of nests (e.g., 15): ")
        or 15
    )
    max_iter = int(input("Enter maximum iterations (e.g., 50): ") or 50)
    pa = float(
        input(
            "Enter discovery rate of alien eggs (pa, default 0.25): "
        )
        or 0.25
    )

    print("\nRunning Cuckoo Search Optimization...")
    best_positions, optimal_energy = cuckoo_search_wsn(
        num_nodes, area_size, sink_pos, pop_size, max_iter, pa
    )

    print("\n--- Optimization Complete ---")
    print(f"Minimum Energy Consumption Found: {optimal_energy:.6f} Joules")
    print("Optimized Sensor Node Coordinates (X, Y):")
```

```
    for i in range(num_nodes):
      print(
          f"  Node {i+1}: ({best_positions[i*2]:.2f}, {best_positions[i*2+1]:.2f})"
      )

  except ValueError:
    print("Invalid input. Please enter numeric values.")
```

```
=== Wireless Sensor Network (WSN) Cuckoo Search Optimization ===
Enter number of sensor nodes (e.g., 10): 6
Enter WSN deployment area side length in meters (e.g., 100): 40
Enter Sink X-coordinate (e.g., 50): 50
Enter Sink Y-coordinate (e.g., 50): 30
Enter Cuckoo Search population size / number of nests (e.g., 15): 20
Enter maximum iterations (e.g., 50): 10
Enter discovery rate of alien eggs (pa, default 0.25): 0.2

Running Cuckoo Search Optimization...
Iteration 1/10 - Best Energy: 0.001391 J
Iteration 2/10 - Best Energy: 0.001391 J
Iteration 4/10 - Best Energy: 0.001391 J
Iteration 6/10 - Best Energy: 0.001378 J
Iteration 8/10 - Best Energy: 0.001378 J
Iteration 10/10 - Best Energy: 0.001378 J

--- Optimization Complete ---
Minimum Energy Consumption Found: 0.001378 Joules
Optimized Sensor Node Coordinates (X, Y):
  Node 1: (35.17, 18.26)
  Node 2: (26.25, 5.20)
  Node 3: (38.07, 13.51)
  Node 4: (25.93, 22.10)
  Node 5: (30.65, 39.70)
  Node 6: (17.40, 11.87)
```